tQuant — Architecture

Andrea Carapelli

November 10, 2026

Contents

1 Overview **3**
1.1 Design principles . . . 3
1.2 Package layout . . . 3
1.3 Automatic differentiation . . . 3

2 Market Data **6**
2.1 Snapshots on disk . . . 6

3 Market Environment **7**
3.1 MarketEnvironment and keys . . . 7
3.2 Market objects and risk factors . . . 7
3.3 The Equity view . . . 8

4 Instruments **9**
4.1 Products, cash flows and indices . . . 9

5 Pricers **10**
5.1 Pricer interface and factory . . . 10

6 Models **11**
6.1 Stochastic processes . . . 11
6.2 Models and processes . . . 11
6.3 Calibration . . . 12

7 Analytics **13**
7.1 Swap exposure . . . 13

8 Foundations **14**
8.1 Dates and conventions . . . 14
8.2 Float64 and dtypes . . . 14
8.3 Performance . . . 14

9 Testing **15**
9.1 Strategy . . . 15

Scope

How the library is built, one chapter per block of the package layout (Figure 1): the market data on disk, the Market Environment (keys, market objects, risk factors), the instruments, the pricers, the models and their calibration, the analytics, the foundations (dates, numerics) and the testing approach. The financial content of the models is in *tQuant — Models and Pricing*.

1 Overview

1.1 Design principles

Prices as TensorFlow graphs, greeks by AAD, risk factors as float64 variables in the market objects, one access point to market data.

To be written in the guided review.

1.2 Package layout

`timehandles`, `numericalhandles`, `markethandles`, `index`, `flows`, `instruments`, `models`, `pricers`, `analytics`, `calibration` (Figure 1).

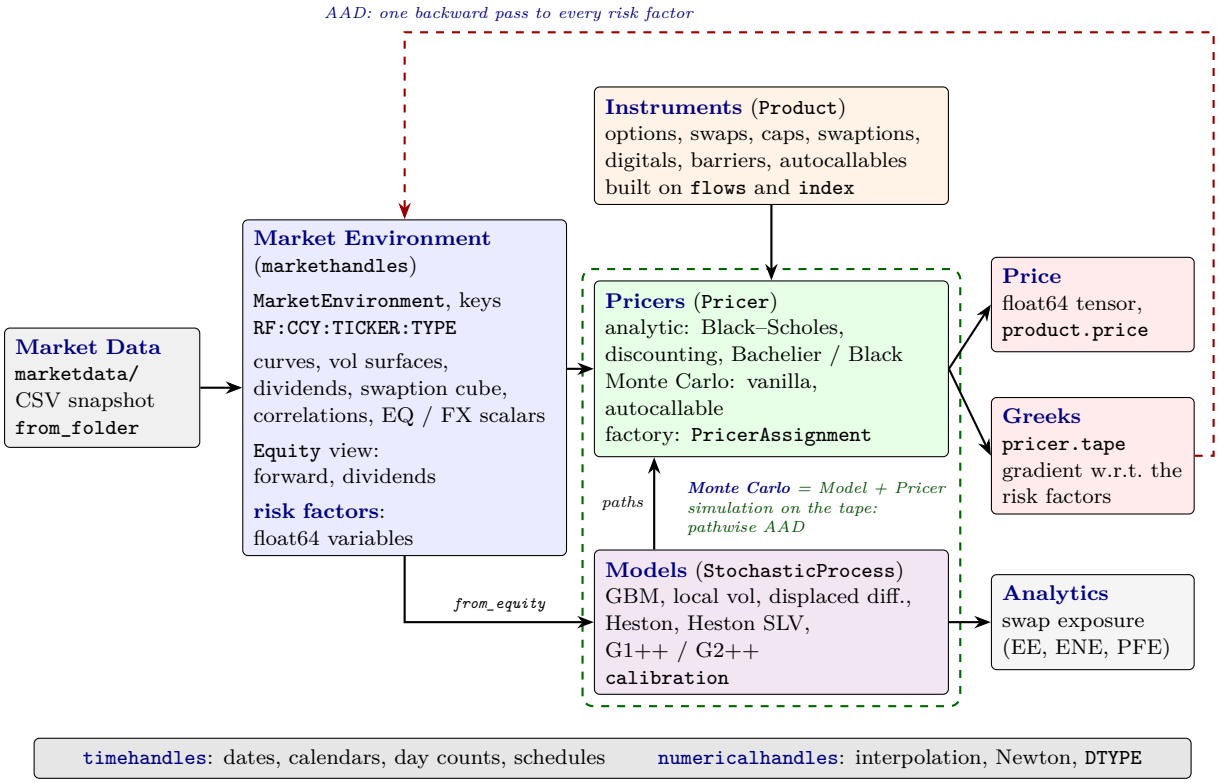


Figure 1: Package layout. Market data enter only through the Market Environment, whose objects hold the risk factors as float64 variables. A pricer turns an instrument and the market into a price graph: in closed form, or (Monte Carlo) by simulating a model built from the same market variables, so the simulated paths are part of the graph. Either way the gradient tape gives the greeks w.r.t. every risk factor in one backward pass. Dates, calendars, interpolation and solvers sit underneath every package.

1.3 Automatic differentiation

Reverse mode. A price is a composition of elementary operations $v_k = \phi_k(v_j, j \prec k)$ from the inputs x (the risk factors) to the output y. The *forward pass* evaluates them and records each operation with the values it needs for its local derivatives (the *tape*). The *backward pass*

walks the tape in reverse and propagates the adjoints $\bar{v}_j = \partial y / \partial v_j$, starting from $\bar{y} = 1$:

$$\bar{v}_j = \sum_{k:\, j \prec k} \bar{v}_k \, \frac{\partial \phi_k}{\partial v_j}, \qquad \frac{\partial y}{\partial x_i} = \bar{x}_i .$$

One backward pass gives the gradient w.r.t. *all* inputs, at a cost of a small multiple of the forward pass (2–5 in practice), whatever their number; bump-and-reprice needs one or two repricings per input. The price is memory: the tape keeps the intermediate values (for a Monte Carlo price, of the order of paths × steps × state). Forward mode (one directional derivative per pass) is the opposite trade-off, fit for few inputs and many outputs. The result is the exact derivative of the computed function (up to rounding), not an approximation: where the code has a kink or a jump, AAD differentiates the code as written.

TensorFlow. The tape is `tf.GradientTape`: it records the operations on the watched tensors, and the trainable variables (`tf.Variable`) are watched automatically. The call `tape.gradient(target, sources)` runs the backward pass and returns the gradients with the structure of `sources` (a dict of lists, as `market_env.risk_factors()`). Mechanics that matter in the library:

- a tape is released after one `gradient` call (`persistent=True` for several); a tape inside another gives second derivatives (gamma); `tape.jacobian` gives the full Jacobian (calibration residuals);
- a source not connected to the target gets `None`, not 0: a value passed through `.numpy()`, `float()`, a comparison or a cast to an integer leaves the graph;
- `tf.gather` on a variable returns sparse `IndexedSlices`: gathers are written to give dense gradients;
- `tf.stop_gradient(v)` keeps the value and cuts the derivative;
- `tf.function` compiles the Python code into a graph, traced once per input signature and differentiable like eager code; the first call pays the tracing.

In tQuant. Every market input is a float64 `tf.Variable` with a stable identity in the market objects (bumps by `assign`), so a gradient w.r.t. `market_env.risk_factors()` is the full bucketed risk. `Pricer.price(..., autodiff=True)` runs `calculate_price` under a tape and keeps it on `pricer.tape`; pricers read the market only through its objects and never detach values. Patterns:

- *Locating, not differentiating*: interval searches (interpolation, vol nodes, kernel bins) act on `stop_gradient` copies; the derivative flows through the interpolation weights.
- *Risk conventions*: a quantity deliberately held fixed is wrapped in `stop_gradient` (the sticky implied yield of the continuous dividend model, the Heston parameters and $\mathbb{E}[v \mid X]$ in Heston SLV).
- *Through a calibration*: the optimiser's iterations are not differentiated. At the optimum θ^* the implicit function theorem gives $\partial\theta/\partial m = -H^{-1}\,\partial^2 L/\partial\theta\,\partial m$; it is injected with a value-preserving step $\theta = \theta^* - (s - \text{stop_gradient}(s))$, $s = H^{-1}\nabla_\theta L(\theta^*, m)$ (H held): the value is θ^*, the derivative the implicit one (Heston, displaced diffusion).
- *Monte Carlo*: the tape over the simulation gives the pathwise derivative of the estimator, valid for Lipschitz payoffs; indicators (digitals, barriers) have no pathwise derivative and are smoothed by a sigmoid, or replaced by a continuous probability (the Brownian bridge of a barrier). Diagnostics such as the standard error are computed on `stop_gradient` values.

The tests check AAD against central finite differences, with common random numbers for Monte Carlo (§9).

2 Market Data

2.1 Snapshots on disk

A snapshot is a folder `marketdata/<yyyymmdd>/<calibration_set>/` (`calibration_set` = time of the snapshot, e.g. `2240`: several snapshots a day) with one CSV per data block; every file is optional. Values are quotes, not objects: the library builds the market objects when loading.

File	Columns
`ir_curves.csv`	`ccy, curve, pillar, value, quote, daycount, interp` (zero rates on pillar dates)
`ir_vol.csv`	`ccy, curve, vol_type, expiry, strike, vol` (caplet vols)
`ir_swaption_vol.csv`	`ccy, curve, expiry, tenor, strike_offset, vol, vol_type`
`eq_spot.csv`	`ticker, ccy, spot, repo, div_yield`
`eq_dividends.csv`	`ticker, ccy, ex_date, pay_date, amount`
`eq_vol.csv`	`ticker, ccy, expiry, strike, vol` (Black vols on absolute strikes)
`fx_spot.csv`, `fx_vol.csv`	`base, quote, spot`; `base, quote, expiry, strike, vol`
`correlations.csv`	`factor_1, factor_2, corr` (pairs; factor = key prefix, e.g. `EQ:EUR:TICKER1`)

`MarketEnvironment.from_folder` (arguments: the root folder, the date and the calibration set) loads the latest snapshot when the last two are `None`; the evaluation date is the folder date. `from_frames` builds the market from the tables, `to_frames` / `to_folder` write it back (exact round trip). The synthetic snapshots come from `tq.generate_market_data` (tables) / `tq.generate_market` (in memory), seeded: Nelson–Siegel OIS curves (EUR, GBP, USD) and EUR 6M as ESTR + spread, SSVI equity and FX smiles, caplet vols and swaption cube from one G2++, 50 tickers with discrete dividends, correlations. [notebook: `market_handles.ipynb`]

3 Market Environment

3.1 MarketEnvironment and keys

`MarketEnvironment` is the only access point to market data (pricers never read files or hold their own copies). It is a flat dictionary with keys `RISKFACTOR:CCY:TICKER:TYPE`: `RISKFACTOR` $\in$ {`IR`, `EQ`, `FX`, `CORR`} (enum `RiskFactor`), `TYPE` $\in$ {`SPOT`, `VOL`, `SWVOL`, `REPO`, `DIVYIELD`, `DIV`} (enum `MarketDataType`); `MarketMapValidator` checks the format and the type of each value.

Key	Value	Accessor
`IR:EUR:ESTR:SPOT`	`RateCurve`	`get_ir_curve(ccy, ticker=None)` (default: OIS)
`IR:EUR:6M:VOL` / `:SWVOL`	caplet `VolatilitySurface` / `SwaptionVolCube`	`get_ir_vol_surface`, `get_swaption_vol_cube`
`EQ:EUR:X:SPOT`, `:REPO`, `:DIVYIELD`	float64 `tf.Variable`	`get_eq_spot`, `get_eq_repo`, `get_eq_div_yield`
`EQ:EUR:X:DIV`	`DividendCurve`	`get_eq_dividends`
`EQ:EUR:X:VOL`	`VolatilitySurface`	`get_eq_vol_surface`; all of them: `get_equity`
`FX:USD:EURUSD:SPOT` / `:VOL`	`tf.Variable` (1 EUR $= x$ USD) / surface	`get_fx_spot`, `get_fx_forward`, `get_fx_vol_surface`
`CORR:ALL:GLOBAL:SPOT`	`CorrelationMatrix`	`get_correlation(names)`

`get_ir_curve(ccy)` without a ticker returns the overnight curve of the currency (ESTR, SOFR, SONIA), the discount curve of everything in that currency; projection curves (e.g. EUR 6M) are asked by ticker. [notebook: `market_handles.ipynb`]

3.2 Market objects and risk factors

Every market input is a float64 `tf.Variable` owned by a market object, with a stable identity (updates by `assign`, so tapes and references stay valid). `market_env.risk_factors()` returns `{key: [variables]}`: one `tape.gradient(price, market_env.risk_factors())` gives every bucketed sensitivity at once.

- `RateCurve`: pillars (dates or year fractions, Act/365 in the snapshots), one variable per pillar zero rate (`rates`, `variables`); `discount`, `zero_rate`, `forward_rate`, `inst_fwd` (definitions: Models and Pricing, Curves). Interpolation `LINEAR` through `numericalhandles.LinearInterp`.
- `VolatilitySurface`: a grid of absolute strikes (`strike`) and maturities in years (`maturity`); the vols are one variable, `volatility_matrix` of shape $[n_T, n_K]$, read by `volatility(strike, tenor)`; `vol_type` is lognormal (equity, FX) or normal (caplets).
- `DividendCurve`: fixed ex-dates and payment dates, the amounts as one variable; PV with a discount curve.
- Scalars (spot, repo, dividend yield, FX spot): one variable each; `MarketEnvironment` wraps a float into a variable.
- `SwaptionVolCube`, `CorrelationMatrix`: one variable each (cube; $[n, n]$ matrix).

[notebook: `market_handles.ipynb`]

3.3 The Equity view

`market_env.get_equity(ticker)` returns an `Equity`: references (not copies) to spot, repo, dividend yield, dividends, vol surface and the discount curve of the currency, plus `currency` and `ticker`. It is the single place for the forward (`forward`, `pv_dividends`, `implied_div_yield`) and the entry point of the model factories (`from_equity`). Dividend model from the data: `DIV` present ⇒ `"discrete"` (`DIVYIELD` ignored), otherwise `"continuous"`. [notebook: `market_handles.ipynb`] [notebook: `equity_underlying.ipynb`]

Member	What it is
`spot`, `repo`, `div_yield`	the market variables (`div_yield` is `None` with a cash schedule; `repo` is a constant 0 when the market has none)
`dividends`	the `DividendCurve`: `ex_dates`, `payment_dates`, `amounts` (one variable), `dividends_before(T)`, `to_dataframe()`
`vol_surface`, `discount_curve`	the market `VolatilitySurface` and the OIS curve of the currency
`year_fraction(T)`	time to a date with the discount-curve day count (Act/365 in the snapshots)
`forward(T)`, `pv_dividends(T)`, `implied_div_yield(T)`	float64 tensors, differentiable w.r.t. the variables above
`risk_factors()`	`{key: [variables]}` restricted to the underlying (spot, repo, dividend yield or amounts, vol, discount curve)
`summary(T)`	table of the data, plus PV of dividends, implied yield, forward and ATM-forward vol at T

`DividendCurve.pv_dividends(T, curve)` sums the amounts with ex-date in $(t_0, T]$, each discounted at its *payment date* (the ex-date when the schedule has no payment dates), with dense weights (a dense gradient on the amounts vector). In the synthetic data the payment is 14 days after the ex-date.

How `BlackScholesPricer` consumes the dividends. Two arguments: `dividend_model` (default `"continuous"`) and `use_implied_repo` (default `True`: the repo enters the carry). With a discrete schedule: `"discrete"` prices on `spot_net` $= S-$ `pv_dividends(T)` with carry = repo; `"continuous"` turns the schedule into q_{eff}, computed with the spot under `stop_gradient`, so the AAD delta is the sticky-yield one (a bump-and-reprice recomputes q_{eff} and gives the sticky-cash delta). Without a schedule the market `DIVYIELD` is used whatever `dividend_model`. The pricer leaves its inputs on the product for inspection: `spot`, `spot_net`, `pv_discrete_dividends`, `repo_margin`, `risk_free_rate`, `discount_factor`, `volatility`, `time_to_maturity` (a variable: theta), `forward`. [notebook: `dividends.ipynb`]

4 Instruments

4.1 Products, cash flows and indices

`Product` and the instruments; `flows/` coupons and legs; generators.

To be written in the guided review.

Equity options derive from `Option`, which holds the option type, the strike (a float64 variable), the exercise type and the underlying. `VanillaOption` is European or American; `DigitalOption` is European, with a `payoff_type` (`CashOrNothing` or `AssetOrNothing`, enum `PayoffType`) and a `cash_amount`. `BlackScholesPricer` prices both, the digitals through `digital_calc` (vol at the strike) or, with `digital_spread` $= h$, as a centred spread of vanillas at $K(1 \pm h)$ on the smile; the factory maps a digital to it. `VanillaMCPricer` prices both too; for the digital payoff `digital_smoothing` sets the sigmoid width (default 1% of the strike, 0 for the hard indicator). [notebook: `digital_options.ipynb`]

The barrier option class, `BarrierOption` in `instruments/barrier.py`, adds the level `barrier`, the type `barrier_type` (an enum `BarrierType` with the members `DownIn`, `DownOut`, `UpIn` and `UpOut`, and the flags `is_down` and `is_in`) and the dates `monitoring_dates`, where `None` means continuous monitoring (property `continuous`). `BlackScholesPricer` prices it through `barrier_calc` (`pricers/black.py`); `VanillaMCPricer` through `_bridge_survival` (continuous) or `_discrete_survival` (monitoring dates merged into the grid).

With the autocall at an unreachable level, an `AutocallableOption` is a strip of digitals (put barrier 0) or a put knocked in at expiry (coupons 0). [notebook: `digital_strip.ipynb`], [notebook: `down_and_in_put.ipynb`]

5 Pricers

5.1 Pricer interface and factory

`Pricer`: `calculate_price`, `price(..., autodiff)`, the tape; `PricerAssignment`.

A pricer implements `calculate_price(product, market_env)`: a TensorFlow graph from the market objects (read only through the `MarketEnvironment` accessors, never copied or detached with `.numpy()`) to a float64 price. `Pricer.price(product, market_env, autodiff=False)` runs it, stores the price on `product.price` and, with `autodiff=True`, keeps the `GradientTape` on `pricer.tape`: the greeks are `pricer.tape.gradient(product.price, variables)` for any market variables (or all of them, `market_env.risk_factors()`); a second tape around the call gives second orders (gamma). The only variable a pricer may create is the time to maturity, left on the product (`product.time_to_maturity`) for theta. `PricerAssignment.create(product)` picks the default pricer by `isinstance`: `VanillaOption` → `BlackScholesPricer`, the rates products to their discounting / Bachelier pricers; Monte Carlo pricers are chosen explicitly. `BlackScholesPricer(dividend_model, use_implied_repo)`: European by the Black formula, American by Bjerksund–Stensland (`pricers/black.py`), inputs from the `Equity` view, diagnostics on the product (Market data, The Equity view). [notebook: `black_scholes.ipynb`]

6 Models

6.1 Stochastic processes

`StochasticProcess` (`size`, `initial_values`, `noise_size`, `evolve`): the interface of everything the Monte Carlo pricers simulate.

Common interface: `StochasticProcess` (`models/stochasticprocess.py`).

- `size()`: state dimension; `initial_values()`; `noise_size()`: normals per path and step (default 1);
- `evolve(t_grid, dw, observation_times=None)`:
  it maps the pre-drawn normals `dw` ($[n_{\text{paths}}, n_{\text{steps}}]$, or with a trailing axis of size `noise_size()`) to paths on the increasing grid `t_grid` (without 0), returning only the `observation_times` columns when given.

The caller draws the normals (`tf.random.stateless_normal` with a seed), which makes the paths reproducible and the bumps common-random-number ones. [notebook: `monte_carlo_gbm.ipynb`]

6.2 Models and processes

Equity model instances (`EquityModel`) separate from the market, bound to an `Equity` into a process.

An equity model (`models/equitymodel.py`) holds *settings* (how it is built) and *calibrated parameters* (float64 `tf.Variables`), never the market: the underlying is given to the calibration and to the binding.

- `Model.calibrate(equity, **settings)`: an instance with the parameters, the fit `report`, the `ticker` and `calibration_date`, and the calibration `jacobian` {parameter : {market key : $[\partial p/\partial$ variable$]$}} with the keys of `Equity.risk_factors()` (computed by `tape.jacobian` through the differentiable calibration); `Model(**parameters)` by hand: no Jacobian;
- `model.process(equity)`: a new `StochasticProcess` (`XxxProcess`, the model is not modified) that rebuilds the market-dependent part (forwards, local vol grid, leverage) and simulates; the closed forms (`vanilla_price`, `implied_vol`) live on the process;
- `Model.from_equity(equity, maturity, strike)`: a process calibrated inside the caller's tape (greeks through the calibration), the factory the pricers use when given the class;
- `model.parameters` (model greeks);
  `par_greeks(model, model_greeks, market_greeks, equity)` ($\partial V/\partial m = \partial V/\partial m|_{\text{direct}} + \sum_p \partial V/\partial p\, \partial p/\partial m$), `to_frames()` / `from_frames()` (settings, parameters, Jacobian in long format, report: exact round trip; JSON values for the settings).

model	settings	parameters	process
`LocalVolatilityModel`	grid, wings, g floor	none (vegas direct)	`LocalVolatilityProcess`
`HestonModel`	martingale correction	$v_0, \kappa, \theta, \xi, \rho$	`HestonProcess`
`DisplacedDiffusionModel`	slice parameters	β, σ per piece	`DisplacedDiffusionProcess`
`HestonSLVModel`	mixing, cap, particles, nodes	$\mathbb{E}[v \mid X]$ table (Heston held)	`HestonSLVProcess`

`HestonSLVModel` composes a `HestonModel` and a `LocalVolatilityModel`; its tables nest theirs (`heston.*`, `local_vol.*`) plus the nodes of the leverage table. `calibrate_leverage(equity)` (the daily step) runs `calibrate_expectation` (particles of its own, normals seeded per row, `tf.function`) and stores $\mathbb{E}[v \mid X]$ (parameter `expectation`, rows $\times$ nodes) and the nodes (`leverage_table`); with `bias_correction` (default) a second run on the same nodes, Richardson-corrected and outside the tape, is stored as `leverage_table["price_values"]` (frame `leverage_price_values`). `process` reads them (one-hot row picks: dense gradient to the table) and `HestonSLVProcess.simulate` returns `X_risk + stop_gradient(X_price - X_risk)`: the value of the corrected table, the derivative of the parameter (smoothing for risk). The calibration and pricing steps are wrapped in `tf.recompute_grad` (check-pointing: only the state $(v, \ln X)$ per step on the tape, the step recomputed in the backward pass; 1Y put with AAD about 8 s instead of 25–40 s). `map_model_greeks` (called by `par_greeks`, which then needs the `equity`) is the adjoint: the calibration re-run on the stored nodes inside a tape, up to the last row with a non-zero $\partial V/\partial \mathbb{E}$, one `tape.gradient` with `output_gradients`; the default `EquityModel.map_model_greeks` uses the stored Jacobian. `HestonSLVModel.from_equity` calibrates Heston and the table inside the pricing tape (same greeks in one pass; tests in `tests/test_slv_vega.py`). `GeometricBrownianMotion` stays a process with an option-specific `from_equity` (vol at (K, T)).

The Monte Carlo pricers (`VanillaMCPricer`, `AutocallableMCPricer`) take a model class or an instance (`check_model`, `bind`): a class is calibrated at each price inside the tape (`from_equity`); an instance is bound with `process(equity)`, its ticker checked against the product's underlying. One backward pass on a nested structure gives model and market greeks:

```
pricer.tape.gradient(price, {"model":  params, "market":  factors}),
```

with `params = model.parameters` and `factors = equity.risk_factors()`. The pricers simulate on a uniform grid ending at the expiry and leave `standard_error`, `discount_factor` and `time_to_maturity` on the product. [notebook: `heston_slv.ipynb`]

6.3 Calibration

Calibration inside the factories (implicit-function greeks), the stored Jacobian, and the rates calibration module.

The least-squares calibrations (Heston, displaced diffusion) run in NumPy / SciPy and end with the implicit step (`_implicit_step`, helpers in `numericalhandles/implicit.py`): at the optimum the gradient g of the objective, its Hessian H and $\partial g/\partial m$ for the calibration inputs m (relative strikes, target prices, weights) are computed once, eagerly, as numbers (`optimality_derivatives`: one backward pass per parameter); `implicit_solution` returns the parameters as a tensor of m with a custom gradient $\mathrm{d}p/\mathrm{d}m = -H^{-1}\,\partial g/\partial m$ (parameters on a bound fixed). The caller's tape never differentiates through the second-order graph of g (first order only). Inside `from_equity` this makes the parameters differentiable functions of the market; `calibrate` takes one Jacobian per parameter to all the risk factors and stores it (`EquityModel.jacobian`), so that the par greeks of an instance equal the greeks through the calibration (`tests/test_model_instances.py`). Calibration of TICKER1: Heston about 3 s (2 s once the COS residuals are compiled), displaced diffusion about 2 s. The rates calibration (`calibration/`) is a separate module (least squares with an AAD Jacobian). *To be written in the guided review.*

7 Analytics

7.1 Swap exposure

`SwapExposureGenerator`: swap MtM on the simulated factors of `GaussianShortRateModel`, EE, ENE, PFE.

To be written in the guided review.

8 Foundations

`timehandles` and `numericalhandles`, used by every package; AAD is in §1.3.

8.1 Dates and conventions

The global evaluation date (`Settings.evaluation_date`), day counts (`DayCounter`), calendars (`Calendar`, `TARGET`), schedules (`ScheduleGenerator`) and the enums of the conventions.

To be written in the guided review.

8.2 Float64 and dtypes

`DTYPE`, `as_variable`, the float32 round trip of `tf.cast` on Python numbers.

To be written in the guided review.

8.3 Performance

Compiled functions traced once, batching, vectorised closed forms.

To be written in the guided review.

9 Testing

9.1 Strategy

Unit tests as the regression check: pinned numbers, independent references (closed forms, parity, finite differences vs AAD, Monte Carlo within standard errors), synthetic self-contained fixtures.

To be written in the guided review.